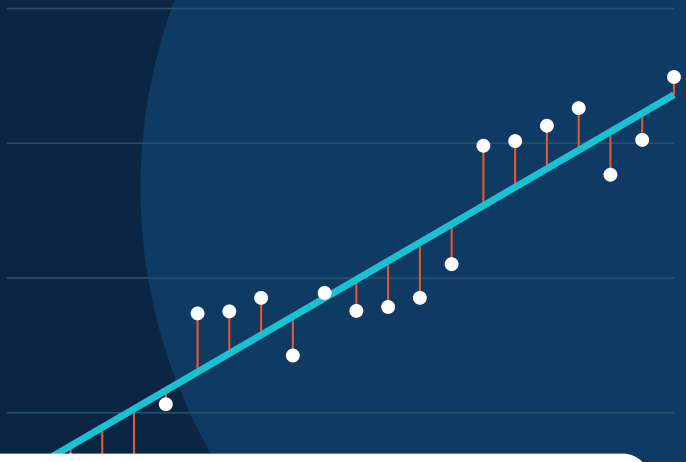
MACHINE LEARNING • STUDY GUIDE

Linear Regression Analysis

From first principles to Ridge, Lasso and ElasticNet, plus 20+ regression models, explained with executed Python code and line-by-line walkthroughs.



PREPARED BY

Chandan Chaudhari

GitHub: github.com/chandanc5525

Contents

1. Introduction 4

1.1 Types of linear regression 4

1.2 Why it matters 4

1.3 Python toolkit used in these notes 5

2. Intuition 5

2.1 The core idea in four steps 5

2.2 Why squared errors? 6

2.3 Three mental pictures 6

3. Mathematics 7

3.1 Model in matrix form 7

3.2 Cost function 7

3.3 Simple regression: closed form 7

3.4 The Normal Equation (derivation) 7

3.5 Gradient Descent (iterative alternative) 8

3.6 Assumptions of linear regression (L.I.N.E. + 1) 9

4. Evaluation Metrics 10

4.1 Error metrics 10

4.2 Goodness-of-fit metrics 10

4.3 Summary table 11

4.4 Worked example 11

4.5 Computing metrics in Python 11

4.6 Diagnosing the assumptions 12

5. Use Cases 13

5.1 What linear regression is best at 13

5.2 When to use and when to avoid 14

5.3 Case study: interpreting a fitted model 14

6. Python Code Structure for Regression Projects 15

6.1 The estimator API 15

6.2 Recommended project workflow 16

6.3 A reusable evaluation helper 16

7. Regularized Linear Models: Ridge, Lasso and ElasticNet 17

7.1 Bias–variance trade-off and the general recipe 17

7.2 Ridge Regression (L2) 18

7.3 Lasso Regression (L1) 18

7.4 ElasticNet 19

8. More Regression Models in the Linear Family 21

8.1 Polynomial Regression 21

8.2 Weighted and Generalized Least Squares 22

8.3 Bayesian Ridge and ARD Regression 23

8.4 Robust Regression: Huber, RANSAC, Theil–Sen 23

8.5 Quantile Regression 24

8.6 LARS, LassoLars and Orthogonal Matching Pursuit 25

8.7 SGDRegressor (Stochastic Gradient Descent) 26

8.8 Principal Component Regression and Partial Least Squares 26

8.9 Generalized Linear Models: Poisson, Gamma and Tweedie 27

8.10 Kernel Ridge Regression 27

8.11 Feature selection: RFE and stepwise methods 28

8.12 Other specialised linear models 29

9. Model Comparison and How to Choose 29

9.1 Decision guide 31

9.2 Master comparison of the penalized family 31

10. Cheat Sheet and Summary 32

Learning roadmap 32

Document map

Sections 1–6 cover introduction, intuition, mathematics, evaluation metrics, use cases and the Python code structure. Sections 7–8 explain Ridge, Lasso, ElasticNet and more than fifteen additional regression models, each with runnable Python code. Section 9 compares all models on one dataset.

1. Introduction

Linear Regression is a **supervised learning** algorithm that models the relationship between a continuous target variable *y* and one or more input features *x* using a straight line (or a flat hyperplane in higher dimensions). It is the oldest, simplest and most interpretable predictive model in statistics and machine learning, and the foundation on which Ridge, Lasso, ElasticNet, GLMs and even neural networks are built.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \varepsilon$$

Symbol	Name	Meaning
y	Target / dependent variable / response	The continuous value we want to predict (price, salary, sales…)
$x_1 \ldots x_p$	Features / independent variables / predictors	Measured inputs used to explain the target
β_0	Intercept (bias)	Predicted value of *y* when every feature equals zero
$\beta_1 \ldots \beta_p$	Coefficients (weights)	Change in *y* for a one-unit change in that feature, holding the others fixed
ε	Error term (noise)	Part of *y* the linear model cannot explain; assumed zero-mean

1.1 Types of linear regression

Type	Description	Example
Simple	One feature, one target	Salary vs years of experience
Multiple	Two or more features, one target	House price vs area, rooms, age, location
Multivariate	Several target variables predicted together	Predict height and weight from age and diet
Polynomial	Linear in parameters, non-linear in features (x^2, x^3…)	Crop yield vs temperature (curved)
Regularized	OLS plus a penalty that shrinks coefficients	Ridge, Lasso, ElasticNet
Robust / Generalized	Handle outliers or non-normal targets	Huber, Quantile, Poisson, Gamma GLM

1.2 Why it matters

- **Interpretable:** every coefficient has a direct business meaning, which is why regulators, doctors and economists still rely on it.
- **Fast and data-efficient:** trains in milliseconds and works with small datasets where deep models would overfit.
- **Strong baseline:** always fit a linear model first; if a complex model cannot beat it, the complexity is not justified.
- **Statistically grounded:** gives confidence intervals, p-values and hypothesis tests, not only predictions.

1.3 Python toolkit used in these notes

Library	Role in a regression project
NumPy	Fast arrays and linear algebra (matrix multiply, solve, norms). Used to build models from scratch.
pandas	Tabular data: loading, cleaning, selecting columns, displaying coefficients.
scikit-learn	All estimators (LinearRegression, Ridge, Lasso…), preprocessing, pipelines, cross-validation, metrics.
statsmodels	Classical statistics: p-values, confidence intervals, diagnostics (Durbin-Watson, Breusch-Pagan), WLS/GLS.
SciPy	Statistical tests such as Shapiro-Wilk for residual normality.
Matplotlib	Plots: fitted lines, residual plots, coefficient paths.

How to read the code in this document

Every Python block is **executed for real** while this PDF is generated, so the printed **OUTPUT** panel is genuine. Blocks share one session in order, so later code re-uses variables from earlier blocks. Each block is followed by a **Code walkthrough** table that explains the lines in plain language. To follow along: `pip install numpy pandas scikit-learn statsmodels scipy matplotlib`.

2. Intuition

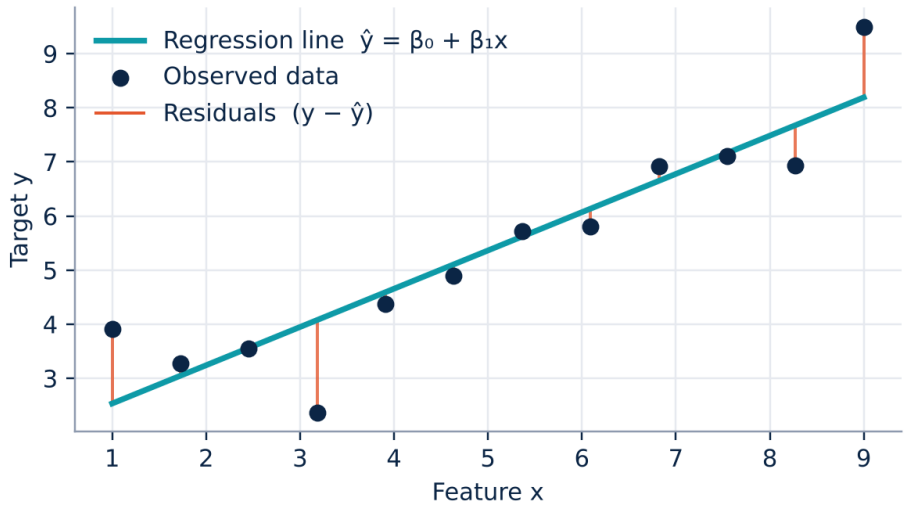


Figure 1. A regression line through noisy data; orange segments are the residuals the line tries to minimize.

Imagine plotting experience against salary for twelve employees. The points are never perfectly on a line, but they follow a trend. Infinitely many lines could be drawn through the cloud; linear regression picks **the one that is closest to all points at the same time**.

2.1 The core idea in four steps

- **Guess** a line (a slope and an intercept).
- **Measure** the vertical gap (residual) between every point and the line.
- **Square and add** the gaps to get one number: the total error.

- **Adjust** slope and intercept to make that number as small as possible. The winning line is the **Ordinary Least Squares (OLS)** solution.

2.2 Why squared errors?

Reason	Explanation
No cancellation	Positive and negative residuals would otherwise cancel and hide a bad fit.
Penalizes large mistakes	An error of 10 costs 100, while ten errors of 1 cost only 10. The line avoids big misses.
Smooth and differentiable	Calculus gives a clean closed-form solution and well-behaved gradients.
Statistical meaning	If noise is Gaussian, minimizing squared error is exactly maximum-likelihood estimation.

2.3 Three mental pictures

- **Slope and intercept:** the slope is how much *y* rises when *x* increases by one unit; the intercept is where the line crosses the y-axis.
- **Springs:** attach every point to the line with a spring. Spring energy grows with the square of the stretch, so the line settles where the total energy is lowest.
- **Geometry:** treat **y** as a vector. The predictions are its **orthogonal projection** onto the space spanned by the feature columns, and the residual vector is perpendicular to every feature.

> **Key insight**
> Because the residual is perpendicular to every feature, the residuals always sum to zero (when an intercept is included) and are uncorrelated with each predictor. You will verify this in the code below.

PYTHON Simple linear regression from scratch with NumPy

```python
import numpy as np
x = np.array([1, 2, 3, 4, 5], dtype=float)   # feature: years of experience
y = np.array([2, 3, 5, 4, 6], dtype=float)   # target: salary (in lakh)
x_bar, y_bar = x.mean(), y.mean()
beta1 = ((x - x_bar) * (y - y_bar)).sum() / ((x - x_bar) ** 2).sum()
beta0 = y_bar - beta1 * x_bar
y_hat = beta0 + beta1 * x
residuals = y - y_hat
print(f'slope={beta1:.2f}  intercept={beta0:.2f}')
print('residuals:', residuals.round(2))
print('sum of residuals:', round(residuals.sum(), 10))
```

OUTPUT

```
slope=0.90  intercept=1.30
residuals: [-0.2 -0.1  1.  -0.9  0.2]
sum of residuals: 0.0
```

Code walkthrough

Lines	What the code does and why
1–3	Import NumPy and create a tiny dataset of five points. `dtype=float` avoids integer division surprises.
4–6	Closed-form OLS for one feature: slope = covariance(x, y) / variance(x); intercept makes the line pass through the point (x̄, ȳ).
7–8	Predict with the fitted line, then compute residuals as actual minus predicted.
9–11	Print the result. The sum of residuals is exactly 0, confirming the geometric property of OLS.

3. Mathematics

3.1 Model in matrix form

For *n* observations and *p* features, stack the data into a design matrix **X** of size *n* × *(p+1)* whose first column is all ones (for the intercept). The whole model becomes one matrix product:

$$\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\beta}, \qquad \mathbf{X} \in \mathbb{R}^{n \times (p+1)},\ \boldsymbol{\beta} \in \mathbb{R}^{(p+1) \times 1}$$

3.2 Cost function

We measure the quality of a parameter vector with the **Residual Sum of Squares (RSS)**, often averaged as the **Mean Squared Error**:

$$J(\boldsymbol{\beta}) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 = \frac{1}{n}(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^T(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$$

3.3 Simple regression: closed form

$$\beta_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2} = \frac{\mathrm{Cov}(x, y)}{\mathrm{Var}(x)} \qquad \beta_0 = \bar{y} - \beta_1 \bar{x}$$

3.4 The Normal Equation (derivation)

Expand the RSS, differentiate with respect to **β**, and set the gradient to zero:

$$\mathrm{RSS} = \mathbf{y}^T\mathbf{y} - 2\boldsymbol{\beta}^T\mathbf{X}^T\mathbf{y} + \boldsymbol{\beta}^T\mathbf{X}^T\mathbf{X}\boldsymbol{\beta} \ \Rightarrow\ \frac{\partial \mathrm{RSS}}{\partial \boldsymbol{\beta}} = -2\mathbf{X}^T(\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) = \mathbf{0}$$

$$\mathbf{X}^T\mathbf{X}\,\boldsymbol{\beta} = \mathbf{X}^T\mathbf{y} \qquad \Longrightarrow \qquad \hat{\boldsymbol{\beta}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

- Requires **$\mathbf{X}^T\mathbf{X}$ to be invertible**: no perfectly collinear features and *n* > *p*.
- Cost of inversion is about $O(p^3)$, so it becomes slow when the number of features is huge.

- Because the cost surface is a convex bowl, this stationary point is the **unique global minimum**.

3.5 Gradient Descent (iterative alternative)

Instead of solving in one shot, start anywhere and repeatedly step downhill along the negative gradient. The learning rate **α** controls the step size.

$$\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} - \alpha \nabla J(\boldsymbol{\beta}), \qquad \nabla J(\boldsymbol{\beta}) = -\tfrac{2}{n}\mathbf{X}^T(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$$

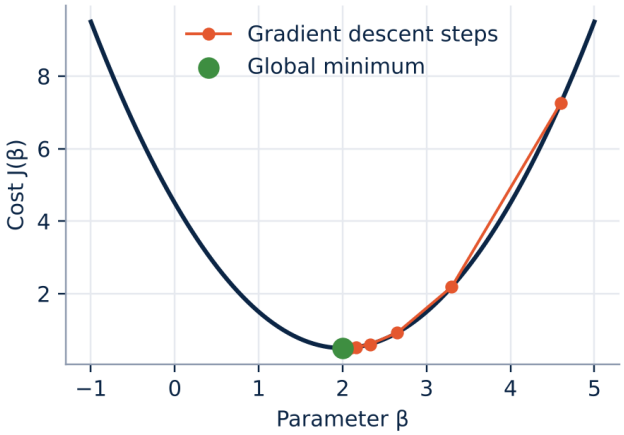


Figure 2. Gradient descent walking down the convex cost bowl toward the global minimum.

	Normal Equation	Gradient Descent
Learning rate α	Not needed	Must be tuned (too small = slow, too large = diverges)
Iterations	None (one calculation)	Many
Large number of features	Slow, memory heavy	Scales well (SGD, mini-batch)
Feature scaling	Not required	Strongly recommended
Typical tool	LinearRegression	SGDRegressor

PYTHON Normal equation vs gradient descent in NumPy

```python
X = np.column_stack([np.ones_like(x), x])   # design matrix with intercept column
beta_ne = np.linalg.solve(X.T @ X, X.T @ y)  # normal equation
beta = np.zeros(2); alpha = 0.05; n = len(y)
for i in range(5001):
    grad = -(2 / n) * X.T @ (y - X @ beta)   # gradient of the MSE
    beta = beta - alpha * grad                # step downhill
    if i % 1000 == 0:
        print(f'iter {i:4d}  beta={beta.round(4)}')
print('normal equation :', beta_ne.round(4))
print('gradient descent :', beta.round(4))
```

OUTPUT

```
iter    0  beta=[0.4  1.38]
iter 1000  beta=[1.3 0.9]
iter 2000  beta=[1.3 0.9]
iter 3000  beta=[1.3 0.9]
iter 4000  beta=[1.3 0.9]
iter 5000  beta=[1.3 0.9]
normal equation : [1.3 0.9]
gradient descent : [1.3 0.9]
```

Code walkthrough

Lines	What the code does and why
1–2	Add a column of ones so that β_0 is learned like any other weight, then solve $X^TX\beta = X^Ty$. `np.linalg.solve` is more stable than computing an explicit inverse.
3–8	Gradient descent: start at zero, compute the gradient from the formula above, update the weights, and print progress every 1000 iterations.
9–10	Both methods reach the same coefficients ($\beta_0 = 1.3$, $\beta_1 = 0.9$), proving they solve the same optimisation problem.

3.6 Assumptions of linear regression (L.I.N.E. + 1)

Assumption	Meaning	How to check	If violated
Linearity	Relationship between X and y is linear in the parameters	Residual vs fitted plot shows no curve	Add polynomial terms, transform variables
Independence	Residuals are not correlated with each other	Durbin-Watson ≈ 2	GLS, time-series models
Normality	Residuals ~ Normal(0, σ^2) (needed for p-values and intervals)	Q-Q plot, Shapiro-Wilk test	Transform y, use robust or GLM models
Equal variance	Constant residual spread (homoscedasticity)	Funnel shape in residual plot, Breusch-Pagan test	Weighted least squares, log-transform y
No multicollinearity	Features are not near-duplicates of each other	VIF > 5–10 is a warning sign	Ridge, drop or combine features, PCA

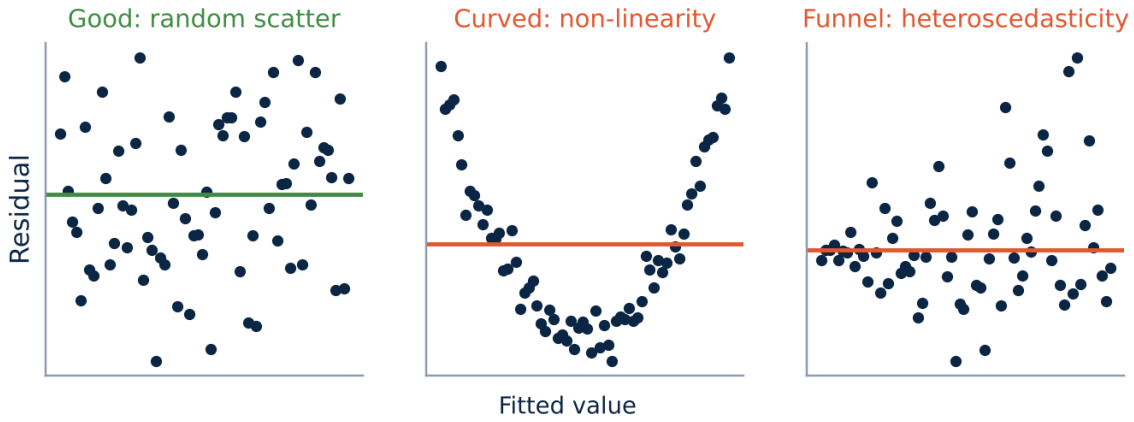


Figure 3. Residual-vs-fitted patterns: only the left panel is healthy.

Gauss–Markov theorem

Under the linearity, independence, zero-mean-error and constant-variance assumptions, OLS is **BLUE**: the Best Linear Unbiased Estimator, with the smallest variance among all linear unbiased estimators. Normality is only needed for exact small-sample inference.

4. Evaluation Metrics

A fitted line is only useful if it predicts **unseen** data well. Always compute metrics on a held-out **test set** or with **cross-validation**, never only on the training data. Let the residual be $e_i = y_i - \hat{y}_i$.

4.1 Error metrics

MAE: Mean Absolute Error

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Average size of the error in the same unit as y. Easy to explain and robust to outliers, because errors are not squared.

MSE and RMSE: Mean Squared Error and its root

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \qquad \mathrm{RMSE} = \sqrt{\mathrm{MSE}}$$

MSE punishes large errors heavily and is the loss that OLS minimizes, but its unit is y^2. RMSE returns to the original unit while keeping the sensitivity to outliers.

4.2 Goodness-of-fit metrics

R^2: Coefficient of Determination

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

The fraction of the variance in y that the model explains. **1** = perfect, **0** = no better than always predicting the mean, and it can become **negative** on test data when the model is worse than that baseline.

Adjusted R^2

$$R^2_{adj} = 1 - \frac{(1 - R^2)(n - 1)}{n - p - 1}$$

Plain R^2 never decreases when a feature is added, even a useless one. Adjusted R^2 penalizes extra features, so it only rises when a new feature genuinely helps.

4.3 Summary table

Metric	Unit	Outlier sensitivity	Better is	Best used for
MAE	Same as y	Low	Lower	Communicating typical error to non-experts
MSE	y^2	High	Lower	Training loss, mathematical analysis
RMSE	Same as y	High	Lower	Comparing models when large errors are costly
R^2	Unitless	Medium	Higher (max 1)	How much variance is explained
Adjusted R^2	Unitless	Medium	Higher	Comparing models with different feature counts
MAPE	Percent	High near y=0	Lower	Relative error (avoid when y can be 0)
AIC / BIC	Unitless	Medium	Lower	Model selection with a complexity penalty

4.4 Worked example

Using the five-point dataset from Section 2 (line $\hat{y} = 1.3 + 0.9x$, $\bar{y} = 4$):

x	y	$\hat{y}$	residual	residual^2	$(y - \bar{y})^2$
1	2	2.2	−0.2	0.04	4
2	3	3.1	−0.1	0.01	1
3	5	4.0	1.0	1.00	1
4	4	4.9	−0.9	0.81	0
5	6	5.8	0.2	0.04	4
Sum			0.0	**1.90**	**10**

MAE = 2.4/5 = **0.48**; **MSE** = 1.9/5 = **0.38**; **RMSE** = √0.38 = **0.616**; **R^2** = 1 − 1.9/10 = **0.81**, so the line explains 81% of the variance.

4.5 Computing metrics in Python

PYTHON Train/test split and all metrics on a real dataset

```python
import pandas as pd
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
data = load_diabetes(as_frame=True)
X, y = data.data, data.target
print('samples, features:', X.shape)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
model = LinearRegression().fit(X_train, y_train)
y_pred = model.predict(X_test)
mae  = mean_absolute_error(y_test, y_pred)
mse  = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2   = r2_score(y_test, y_pred)
n, p = X_test.shape
adj_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1)
print(f'MAE={mae:.2f}  MSE={mse:.1f}  RMSE={rmse:.2f}')
```

```
20 print(f'R2={r2:.3f}  Adjusted R2={adj_r2:.3f}')
```

OUTPUT

```
samples, features: (442, 10)
MAE=42.79  MSE=2900.2  RMSE=53.85
R2=0.453  Adjusted R2=0.382
```

Code walkthrough

Lines	What the code does and why
1–5	Import the dataset loader, the splitter, the model and the metric functions. All of them live in `scikit-learn`.
6–8	Load the Diabetes dataset (442 patients, 10 clinical features, target = disease progression after one year) as pandas objects.
9–10	Hold out 20% of rows as a test set. `random_state` makes the split reproducible.
11–12	**Fit** learns β from the training data only (normal equation internally); **predict** applies it to unseen rows.
13–18	Compute each metric with the sklearn helper functions; RMSE is the square root of MSE; adjusted R^2 is calculated from the formula above.
19–20	Report the results. $R^2 \approx 0.45$ means the ten features explain about 45% of the variation in disease progression.

4.6 Diagnosing the assumptions

PYTHON Statistical summary and assumption checks with statsmodels

```
1  import statsmodels.api as sm
2  ols = sm.OLS(y_train, sm.add_constant(X_train)).fit()
3  print(ols.summary().tables[1])
4  from statsmodels.stats.stattools import durbin_watson
5  from statsmodels.stats.diagnostic import het_breuschpagan
6  from statsmodels.stats.outliers_influence import variance_inflation_factor
7  from scipy import stats
8  Xc = sm.add_constant(X_train)
9  print('Durbin-Watson   :', round(durbin_watson(ols.resid), 2), '(about 2 = independent)')
10 print('Breusch-Pagan p :', round(het_breuschpagan(ols.resid, Xc)[1], 3), '(>0.05 = equal
       variance)')
11 print('Shapiro-Wilk p  :', round(stats.shapiro(ols.resid)[1], 3), '(>0.05 = normal
       residuals)')
12 vif = {c: round(float(variance_inflation_factor(Xc.values, i)), 1) for i, c in
       enumerate(Xc.columns) if c != 'const'}
13 print('VIF:', vif)
```

OUTPUT

```
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const        151.3456      2.902     52.155      0.000     145.638     157.053
age           37.9040     69.056      0.549      0.583     -97.923     173.731
sex         -241.9644     68.570     -3.529      0.000    -376.836    -107.093
bmi          542.4288     76.956      7.049      0.000     391.062     693.795
bp           347.7038     71.357      4.873      0.000     207.350     488.057
s1          -931.4888    451.138     -2.065      0.040   -1818.844     -44.134
s2           518.0623    364.114      1.423      0.156    -198.122    1234.247
s3           163.4200    233.014      0.701      0.484    -294.901     621.741
s4           275.3179    185.400      1.485      0.138     -89.349     639.985
s5           736.1989    192.437      3.826      0.000     357.689    1114.709
s6            48.6707     73.435      0.663      0.508     -95.771     193.113
==============================================================================
Durbin-Watson   : 1.79 (about 2 = independent)
Breusch-Pagan p : 0.035 (>0.05 = equal variance)
Shapiro-Wilk p  : 0.64 (>0.05 = normal residuals)
VIF: {'age': 1.2, 'sex': 1.3, 'bmi': 1.6, 'bp': 1.4, 's1': 55.3, 's2': 35.8, 's3': 14.3, 's4': 9.3,
```

Code walkthrough

Lines	What the code does and why
1–3	statsmodels needs the intercept column added explicitly (`add_constant`). The summary table lists each coefficient, its standard error, t-statistic, p-value and 95% confidence interval.
4–7	Import the diagnostic tests: independence (Durbin-Watson), equal variance (Breusch-Pagan), multicollinearity (VIF) and normality (Shapiro-Wilk from SciPy).
8–13	Run each test on the residuals. A very small p-value rejects the assumption. VIF values above 10 (here s1, s2, s3 and about 10 for s5) reveal strong multicollinearity, which motivates the regularized models in Section 7.

Reading the diagnostics

The very high VIF for **s1** (≈59) and **s2** (≈39) means those two blood-serum features carry almost the same information, so their individual coefficients are unstable. This is exactly the situation where **Ridge** or **ElasticNet** beats plain OLS in stability.

5. Use Cases

Domain	Typical target (y)	Typical features (x)
Real estate	House price	Area, rooms, location, age of building
Marketing	Sales / revenue	TV, radio, social-media ad spend
Finance	Stock return (CAPM beta), credit loss	Market return, risk factors, ratios
Healthcare	Blood pressure, drug dose, disease progression	Age, BMI, lab values, lifestyle
Economics	Demand, GDP, inflation	Price, income, interest rate
Agriculture	Crop yield	Rainfall, fertilizer, temperature
Energy	Electricity load forecast	Temperature, hour of day, holidays
Insurance	Premium / claim amount	Age, history, vehicle or health profile
Manufacturing	Product quality, defect rate	Temperature, pressure, machine speed
Sports analytics	Player performance	Training load, past statistics
E-commerce / Logistics	Delivery time, demand	Distance, traffic, time, season
Education	Exam score	Study hours, attendance

5.1 What linear regression is best at

- **Forecasting** trends such as next quarter's sales or tomorrow's electricity demand.
- **Explaining** relationships: how much does one extra rupee of advertising change sales?
- **Feature importance and hypothesis testing** in research, using p-values and confidence intervals.
- **Baseline modelling** before trying random forests, gradient boosting or neural networks.

5.2 When to use and when to avoid

Use it when	Avoid or upgrade when
The relationship is roughly linear	Strongly non-linear patterns (use polynomial, trees, boosting)
You need interpretability and explanations	The target is categorical (use logistic regression or classifiers)
Data is limited	Heavy outliers (use Huber, RANSAC, quantile regression)
You need a fast, reliable baseline	Features are highly correlated (use Ridge or ElasticNet)
Errors are roughly normal with constant variance	Target is a count or strictly positive (use Poisson or Gamma GLM)

Common pitfalls

Correlation is not causation. A large coefficient shows association, not that changing the feature will change the target.
Extrapolation is dangerous. The line is only trustworthy inside the range of the training data.
Data leakage. Fit scalers and imputers on the training split only; use a Pipeline.

5.3 Case study: interpreting a fitted model

The pipeline below standardizes all features (so coefficients are comparable), measures generalization with 5-fold cross-validation and lists which features drive the prediction.

PYTHON Case study: scaled pipeline, cross-validation and coefficient table

```python
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score, KFold
pipe = make_pipeline(StandardScaler(), LinearRegression())
cv = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(pipe, X_train, y_train, cv=cv, scoring='r2')
print('CV R2 per fold:', scores.round(3))
print(f'mean={scores.mean():.3f}  std={scores.std():.3f}')
pipe.fit(X_train, y_train)
coef = pd.Series(pipe[-1].coef_, index=X.columns)
print(coef.sort_values(key=abs, ascending=False).round(2))
print('intercept:', round(pipe[-1].intercept_, 2))
```

OUTPUT

```
CV R2 per fold: [0.47  0.537 0.411 0.491 0.493]
mean=0.480  std=0.041
s1    -44.45
s5     35.16
bmi    25.61
s2     24.64
bp     16.83
s4     13.14
sex   -11.51
s3      7.68
s6      2.35
age     1.75
dtype: float64
intercept: 153.74
```

Code walkthrough

Lines	What the code does and why
1–3	Pipeline chains preprocessing and the model so that scaling parameters are learned from training folds only (no leakage). KFold defines the cross-validation splits.
4–6	Build the pipeline, create a shuffled 5-fold splitter and compute R^2 on each validation fold. Each fold re-fits the whole pipeline.
7–8	The mean estimates expected performance; the standard deviation shows how stable the model is across different data subsets.
9–12	Fit on the full training set and read the learned weights from the last pipeline step (`pipe[-1]`). Sorting by absolute value ranks the weights: s1 and s5 (log serum triglycerides) and BMI are largest. The large opposite weights of s1 and s2 partly cancel because they are collinear, so BMI and s5 are the more trustworthy drivers.

6. Python Code Structure for Regression Projects

Every scikit-learn model follows the same small interface, which is why you can swap Ridge for Lasso by changing one line. Understanding this structure is the key to writing clean, leak-free machine-learning code.

6.1 The estimator API

Step / member	What it does	Example
Instantiate	Create the model and set hyper-parameters (nothing is learned yet)	`m = Ridge(alpha=1.0)`
.fit(X, y)	Learn the parameters from training data; returns the model itself	`m.fit(X_train, y_train)`
.predict(X)	Produce predictions for new rows	`m.predict(X_test)`
.score(X, y)	Default metric; R^2 for regressors	`m.score(X_test, y_test)`
.coef_ / .intercept_	Learned weights (attributes ending in _ exist only after fit)	`m.coef_`
.get_params() / .set_params()	Read or change hyper-parameters; used by grid search	`m.set_params(alpha=5)`
.transform(X)	Transformers only (scalers, PCA): apply a learned transformation	`scaler.transform(X)`

6.2 Recommended project workflow

#	Stage	Python tool	Notes
1	Load and inspect data	pandas: read_csv, describe, info, isna	Look for missing values, outliers, units
2	Split first	train_test_split	Split *before* any fitting to avoid leakage
3	Preprocess	Pipeline + StandardScaler, OneHotEncoder, SimpleImputer	Put every step inside a pipeline
4	Fit and tune	GridSearchCV or the *CV estimators (RidgeCV...)	Choose hyper-parameters with cross-validation
5	Evaluate	metrics + residual plots + diagnostics	Judge on the untouched test set once
6	Interpret and deploy	coef_, joblib.dump	Save the whole pipeline, not just the model

6.3 A reusable evaluation helper

PYTHON Reusable function to compare any regression model

```python
def evaluate(name, model):
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    return {'model': name,
            'R2': r2_score(y_test, pred),
            'RMSE': np.sqrt(mean_squared_error(y_test, pred)),
            'MAE': mean_absolute_error(y_test, pred)}
from sklearn.linear_model import Ridge
rows = [evaluate('OLS',   make_pipeline(StandardScaler(), LinearRegression())),
        evaluate('Ridge', make_pipeline(StandardScaler(), Ridge(alpha=10)))]
print(pd.DataFrame(rows).round(3).to_string(index=False))
```

OUTPUT

```
model    R2   RMSE    MAE
  OLS 0.453 53.853 42.794
Ridge 0.457 53.626 42.857
```

Code walkthrough

Lines	What the code does and why
1–7	A function that fits any estimator, predicts on the test set and returns a dictionary of metrics. Because all sklearn models share `fit` and `predict`, the same function works for every model in this document.
8–10	Call the helper for two models wrapped in pipelines. Adding more models only requires adding more lines to this list.
11	Convert the list of dictionaries to a DataFrame for a tidy comparison table.

Suggested project folder layout for a professional repository:

```
linear-regression-project/
|-- data/            raw and processed datasets (never edit raw files)
|-- notebooks/       exploration: 01_eda.ipynb, 02_modeling.ipynb
|-- src/
|   |-- features.py  cleaning and feature engineering functions
|   |-- train.py     builds the Pipeline, runs cross-validation, saves model
|   `-- evaluate.py  metrics, residual plots, diagnostics
|-- models/          saved pipelines (joblib)
|-- requirements.txt pinned library versions
`-- README.md        problem, results, how to run
```

7. Regularized Linear Models: Ridge, Lasso and ElasticNet

Plain OLS can **overfit** when there are many features, correlated features or little data: coefficients grow huge and swing wildly with small changes in the data. **Regularization** adds a penalty on the size of the coefficients, trading a little bias for a large reduction in variance.

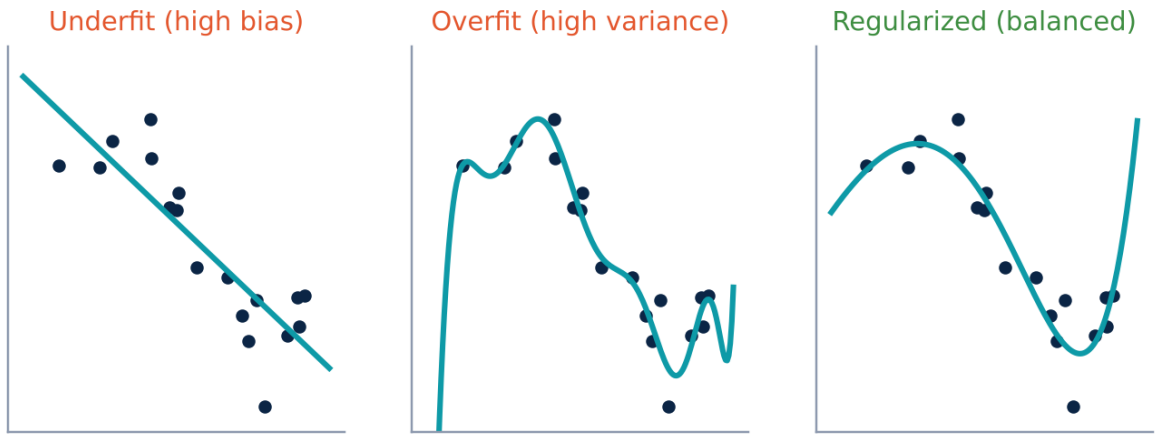


Figure 4. Underfitting, overfitting and a regularized fit of the same noisy data (degree-15 polynomial).

7.1 Bias–variance trade-off and the general recipe

- **High bias (underfit):** model too simple, large error on both training and test data.
- **High variance (overfit):** model too flexible, tiny training error but large test error.
- **Regularization** deliberately accepts slightly biased estimates to obtain much lower variance.

$$J_{reg}(\boldsymbol{\beta}) = \sum_i (y_i - \hat{y}_i)^2 + \lambda \cdot \text{Penalty}(\boldsymbol{\beta})$$

Rule	Why it matters
Standardize features first	The penalty is scale-sensitive: a feature measured in large units would be penalized unfairly.
Do not penalize the intercept	The intercept only shifts predictions; sklearn handles this automatically.
Tune λ with cross-validation	λ (called `alpha` in scikit-learn) controls strength: 0 = OLS, very large = all coefficients near 0.

Model	Penalty	Effect on coefficients	Feature selection?
Ridge	L2: λ Σβ²	Shrinks all smoothly, never exactly zero	No
Lasso	L1: λ Σ|β|	Shrinks and sets some exactly to zero	Yes
ElasticNet	Mix of L1 and L2	Sparse yet stable with correlated features	Yes

7.2 Ridge Regression (L2)

$$J(\boldsymbol{\beta}) = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda\sum_{j=1}^{p}\beta_j^2 \qquad \hat{\boldsymbol{\beta}}_{ridge} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y}$$

- Adding λI makes the matrix **always invertible**, which cures multicollinearity.
- Shrinks coefficients **toward zero but never exactly to zero**, so every feature is kept.
- Correlated features receive **similar, shared** weights, which makes predictions stable.
- λ → 0 gives OLS; λ → ∞ drives all coefficients to 0 (the model predicts the mean).
- **Bayesian view:** the MAP estimate under a Gaussian prior on the weights.
- **Best when** many features each carry a little signal, or features are strongly correlated.

PYTHON Ridge with cross-validated alpha

```
1  from sklearn.linear_model import Ridge, RidgeCV
2  alphas = np.logspace(-3, 3, 50)   # 50 candidate strengths from 0.001 to 1000
3  ridge = make_pipeline(StandardScaler(), RidgeCV(alphas=alphas, cv=5))
4  ridge.fit(X_train, y_train)
5  print('best alpha:', round(ridge[-1].alpha_, 3))
6  print('test R2   :', round(r2_score(y_test, ridge.predict(X_test)), 3))
7  for a in [0.01, 1, 100, 1000]:
8      m = make_pipeline(StandardScaler(), Ridge(alpha=a)).fit(X_train, y_train)
9      print(f'alpha={a:<7} coefficient norm = {np.linalg.norm(m[-1].coef_):6.2f}')
```

OUTPUT

```
best alpha: 44.984
test R2   : 0.461
alpha=0.01    coefficient norm =  71.48
alpha=1       coefficient norm =  60.91
alpha=100     coefficient norm =  34.28
alpha=1000    coefficient norm =  15.88
```

Code walkthrough

Lines	What the code does and why
1–4	`RidgeCV` tries every alpha with 5-fold cross-validation and keeps the best. Scaling is inside the pipeline so the penalty treats all features equally.
5–6	`alpha_` is the winning strength. Evaluate once on the untouched test data.
7–9	Demonstrate shrinkage: as alpha increases, the overall size (L2 norm) of the coefficient vector steadily falls.

7.3 Lasso Regression (L1)

Lasso stands for Least Absolute Shrinkage and Selection Operator.

$$J(\boldsymbol{\beta}) = \frac{1}{2n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda\sum_{j=1}^{p}|\beta_j|$$

- The L1 penalty has sharp corners, so many coefficients become **exactly zero**: automatic **feature selection** and sparse, simple models.
- **No closed form** because |β| is not differentiable at 0; solved by **coordinate descent** with soft-thresholding.
- **Bayesian view:** the MAP estimate under a Laplace prior.

- **Limitations:** among a group of correlated features it tends to keep one arbitrarily; if p > n it can select at most n features.
- **Best when** you believe only a few features truly matter (high-dimensional data, genomics, text).

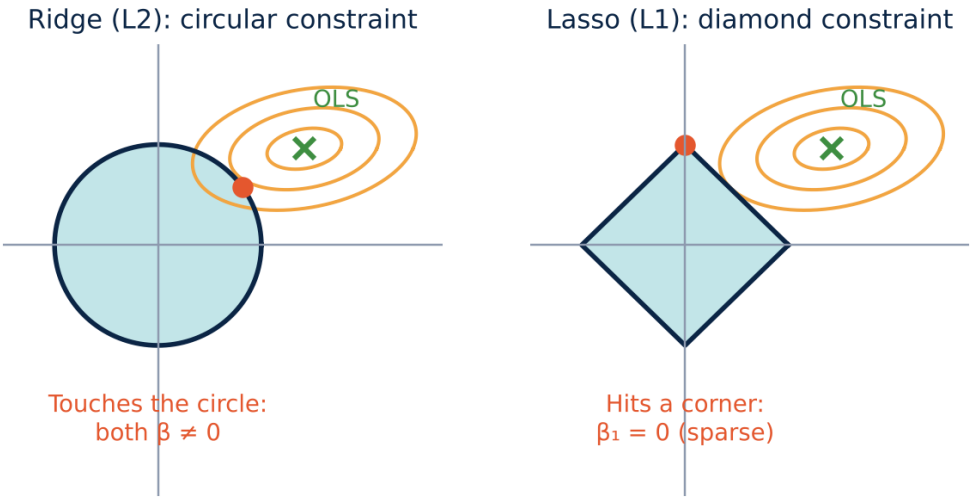


Figure 5. Why Lasso produces zeros: the diamond constraint touches the error contours at a corner on an axis.

PYTHON Lasso feature selection

```
1  from sklearn.linear_model import LassoCV
2  lasso = make_pipeline(StandardScaler(), LassoCV(cv=5, random_state=0))
3  lasso.fit(X_train, y_train)
4  coef = pd.Series(lasso[-1].coef_, index=X.columns)
5  print('best alpha:', round(lasso[-1].alpha_, 3))
6  print('kept   :', list(coef[coef != 0].index))
7  print('dropped:', list(coef[coef == 0].index))
8  print('test R2:', round(r2_score(y_test, lasso.predict(X_test)), 3))
```

OUTPUT

```
best alpha: 1.655
kept   : ['sex', 'bmi', 'bp', 's1', 's3', 's5', 's6']
dropped: ['age', 's2', 's4']
test R2: 0.471
```

Code walkthrough

Lines	What the code does and why
1–3	`LassoCV` automatically builds a grid of alphas, evaluates each with 5-fold CV and refits with the best one.
4–7	Wrap the coefficients in a Series to see names. Features whose coefficient is exactly 0 have been removed from the model.
8	Despite using fewer features, Lasso generalizes slightly better than OLS on this split.

7.4 ElasticNet

ElasticNet combines both penalties to obtain sparsity from Lasso and stability from Ridge.

$$J(\boldsymbol{\beta}) = \frac{1}{2n}\sum_{i}(y_i - \hat{y}_i)^2 + \lambda\left[\rho\sum_{j}|\beta_j| + \frac{1-\rho}{2}\sum_{j}\beta_j^2\right]$$

- **ρ** (`l1_ratio` in sklearn): ρ = 1 is Lasso, ρ = 0 is Ridge, values in between mix them.
- **Grouping effect:** correlated features are kept or dropped together instead of arbitrarily choosing one.

- Works when $p > n$ and is not limited to selecting n features.
- Two hyper-parameters (alpha and l1_ratio) are tuned jointly with `ElasticNetCV`.

PYTHON ElasticNet with cross-validated alpha and l1_ratio

```python
from sklearn.linear_model import ElasticNetCV
enet = make_pipeline(StandardScaler(),
    ElasticNetCV(l1_ratio=[.1, .5, .7, .9, .95, 1], cv=5, random_state=0))
enet.fit(X_train, y_train)
print('best alpha   :', round(enet[-1].alpha_, 3))
print('best l1_ratio:', enet[-1].l1_ratio_)
print('zero coefs   :', int((enet[-1].coef_ == 0).sum()))
print('test R2      :', round(r2_score(y_test, enet.predict(X_test)), 3))
```

OUTPUT

```
best alpha   : 0.189
best l1_ratio: 0.5
zero coefs   : 0
test R2      : 0.461
```

Code walkthrough

Lines	What the code does and why
1–4	Search six mixing ratios (and an automatic alpha grid for each) with 5-fold cross-validation.
5–8	Report the chosen hyper-parameters, how many features were eliminated and the test R^2.

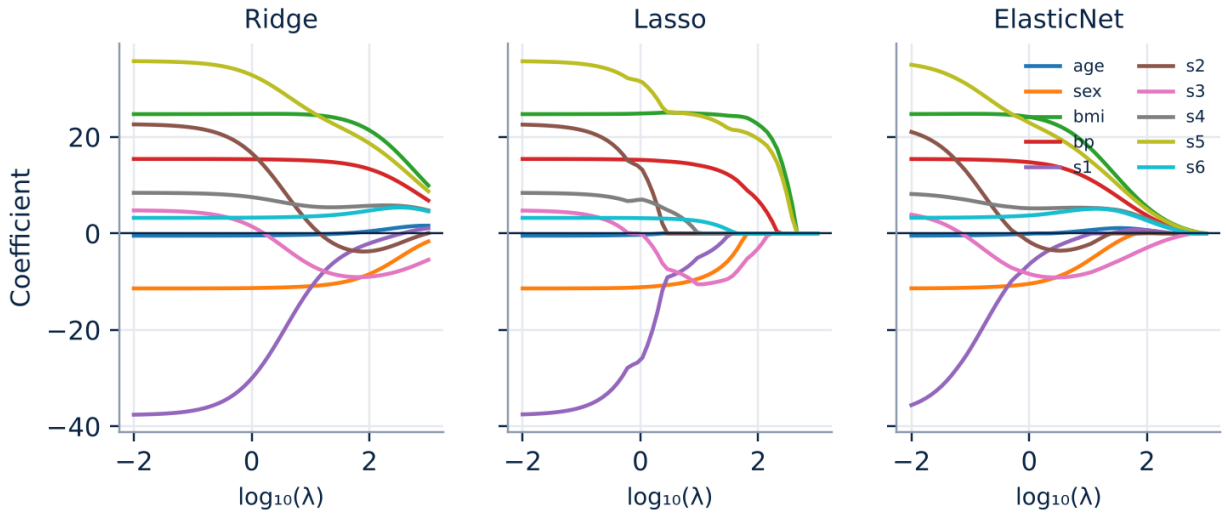


Figure 6. Coefficient paths on the Diabetes data as λ grows: Ridge shrinks smoothly; Lasso and ElasticNet hit exactly zero.

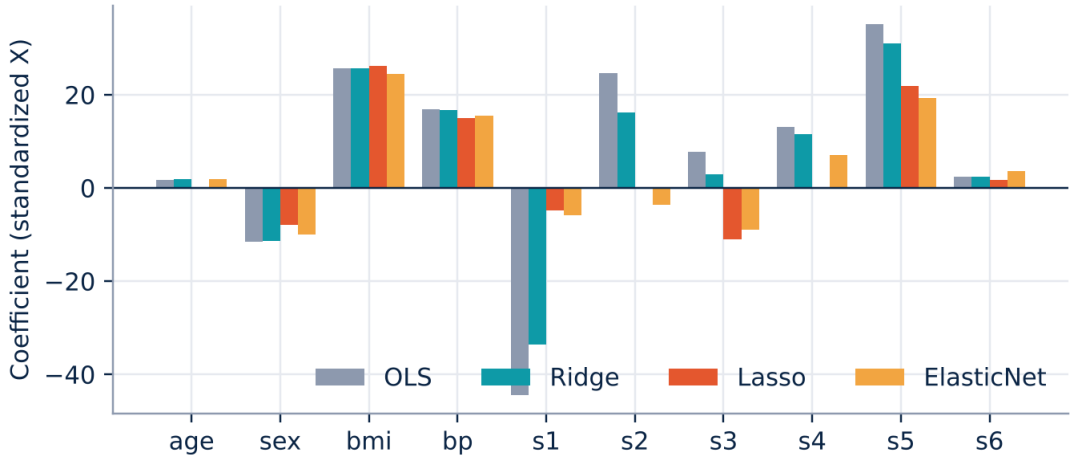


Figure 7. Fitted coefficients of OLS, Ridge, Lasso and ElasticNet on the Diabetes dataset (standardized features).

Notice how OLS assigns large opposite-signed weights to the correlated pair **s1** and **s2**, Ridge softens them, and Lasso removes s2 entirely.

8. More Regression Models in the Linear Family

Beyond Ridge, Lasso and ElasticNet there is a rich family of linear-type models. Each solves a specific weakness of OLS: curved relationships, unequal variance, outliers, tiny data, huge data, many correlated features or non-normal targets. All examples reuse the Diabetes training data defined earlier.

Problem with OLS	Model to reach for	Section
Curved relationship	Polynomial regression, Kernel Ridge	8.1, 8.10
Non-constant variance / correlated errors	Weighted LS, Generalized LS	8.2
Need uncertainty or automatic feature relevance	Bayesian Ridge, ARD	8.3
Outliers in the target	Huber, RANSAC, Theil-Sen	8.4
Want percentiles, not the mean	Quantile regression	8.5
Many features, need sparse model fast	LARS, LassoLars, Orthogonal Matching Pursuit	8.6
Very large dataset	SGDRegressor	8.7
Many correlated features	PCR, PLS regression	8.8
Non-normal or positive target	Poisson, Gamma, Tweedie GLM	8.9
Choose a subset of features	RFE / stepwise selection	8.11

8.1 Polynomial Regression

Create new features such as x^2, x^3 and interaction terms x_1x_2, then fit an ordinary linear model on them. The model is still **linear in the parameters** β, so all OLS theory applies, but the fitted curve can bend.

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_d x^d$$

- Degree *d* is the key hyper-parameter: too low underfits, too high overfits (see Figure 4). Combine with Ridge to tame the large coefficients.

PYTHON Polynomial features plus Ridge

```
from sklearn.preprocessing import PolynomialFeatures
poly = make_pipeline(StandardScaler(),
    PolynomialFeatures(degree=2, include_bias=False),
    StandardScaler(), Ridge(alpha=50)).fit(X_train, y_train)
print('features after expansion:', poly[1].n_output_features_)
print('test R2:', round(r2_score(y_test, poly.predict(X_test)), 3))
```

OUTPUT

```
features after expansion: 65
test R2: 0.51
```

Code walkthrough

Lines	What the code does and why
1–4	Scale, expand to all squares and pairwise products, rescale the new features and fit a strongly regularized Ridge to keep the 65 features under control.
5–6	Ten original features become 65 (10 linear + 10 squares + 45 interactions). Test R^2 improves over plain OLS, showing mild non-linearity.

8.2 Weighted and Generalized Least Squares

Weighted Least Squares (WLS) multiplies each squared residual by a weight, usually the inverse of that observation's variance, so noisy points count less. **Generalized Least Squares (GLS)** goes further and models the full covariance matrix Σ of the errors, which handles correlated errors such as autocorrelation in time series.

$$\hat{\boldsymbol{\beta}}_{WLS} = (\mathbf{X}^T\mathbf{W}\mathbf{X})^{-1}\mathbf{X}^T\mathbf{W}\mathbf{y} \qquad \hat{\boldsymbol{\beta}}_{GLS} = (\mathbf{X}^T\boldsymbol{\Sigma}^{-1}\mathbf{X})^{-1}\mathbf{X}^T\boldsymbol{\Sigma}^{-1}\mathbf{y}$$

PYTHON Feasible WLS and AR(1) GLS with statsmodels

```
abs_res = np.abs(ols.resid)
var_fit = sm.OLS(abs_res, sm.add_constant(ols.fittedvalues)).fit().fittedvalues
w = 1 / np.clip(var_fit, 1, None) ** 2
wls = sm.WLS(y_train, sm.add_constant(X_train), weights=w).fit()
print('std error of bmi  OLS:', round(ols.bse['bmi'], 2), '  WLS:', round(wls.bse['bmi'],
    2))
gls = sm.GLSAR(y_train, sm.add_constant(X_train), rho=1).iterative_fit(maxiter=5)
print('estimated AR(1) rho:', np.round(gls.model.rho, 3))
```

OUTPUT

```
std error of bmi  OLS: 76.96   WLS: 78.68
estimated AR(1) rho: [0.097]
```

Code walkthrough

Lines	What the code does and why
1–3	Estimate how the residual spread depends on the prediction (regress |residual| on fitted values), then set each weight to 1 / variance.
4–5	Fit WLS with those weights and compare the standard error of the BMI coefficient. The values differ because WLS corrects for unequal variance (the Breusch-Pagan test in Section 4.6 flagged mild heteroscedasticity, p = 0.035), so its standard errors give more trustworthy inference.
6–7	GLSAR alternates between estimating the error autocorrelation ρ and refitting. This data is not a time series, so ρ is near 0; on real sequential data it would capture serial correlation.

8.3 Bayesian Ridge and ARD Regression

Bayesian linear regression places probability distributions on the weights and the noise level, and learns their precision from the data (no manual λ). The prediction is a **distribution**, giving an uncertainty estimate for every point. **ARD (Automatic Relevance Determination)** gives each feature its own precision, automatically pruning irrelevant features, similar in spirit to Lasso.

$$p(\boldsymbol{\beta} \mid \mathbf{X}, \mathbf{y}) \;\propto\; p(\mathbf{y} \mid \mathbf{X}, \boldsymbol{\beta})\, p(\boldsymbol{\beta}), \qquad p(\boldsymbol{\beta}) = \mathcal{N}(\mathbf{0},\; \lambda^{-1}\mathbf{I})$$

PYTHON Predictive uncertainty and automatic feature pruning

```python
from sklearn.linear_model import BayesianRidge, ARDRegression
br = make_pipeline(StandardScaler(), BayesianRidge()).fit(X_train, y_train)
mean, std = br.predict(X_test, return_std=True)
print('test R2:', round(r2_score(y_test, mean), 3))
print('average predictive std:', round(std.mean(), 1))
ard = make_pipeline(StandardScaler(), ARDRegression()).fit(X_train, y_train)
print('ARD near-zero coefficients:', int((np.abs(ard[-1].coef_) < 1e-3).sum()))
```

OUTPUT

```
test R2: 0.458
average predictive std: 55.1
ARD near-zero coefficients: 1
```

Code walkthrough

Lines	What the code does and why
1–3	Fit Bayesian Ridge. With `return_std=True` predict returns both the mean prediction and its standard deviation.
4–5	Accuracy is similar to Ridge, but now each prediction comes with an uncertainty band (about ±2 std for 95%).
6–7	ARD learns one precision per feature; coefficients of useless features are driven to (almost) zero.

8.4 Robust Regression: Huber, RANSAC, Theil–Sen

OLS squares the residuals, so a few extreme outliers can drag the whole line toward them. Robust estimators limit that influence.

Model	Idea	Strength	Weakness
Huber	Quadratic loss for small residuals, linear loss for large ones	Smooth, fast, keeps all data	Only protects against outliers in y
RANSAC	Repeatedly fits on random subsets and keeps the model with most inliers	Handles up to ~50% contamination	Random, needs a good inlier threshold
Theil–Sen	Median of slopes over many subsets of points	Very robust, no tuning	Slow for large n or many features

$$L_\delta(e) = \tfrac{1}{2}e^2 \;\; \text{if } |e| \le \delta, \qquad L_\delta(e) = \delta\big(|e| - \tfrac{1}{2}\delta\big) \;\; \text{if } |e| > \delta$$

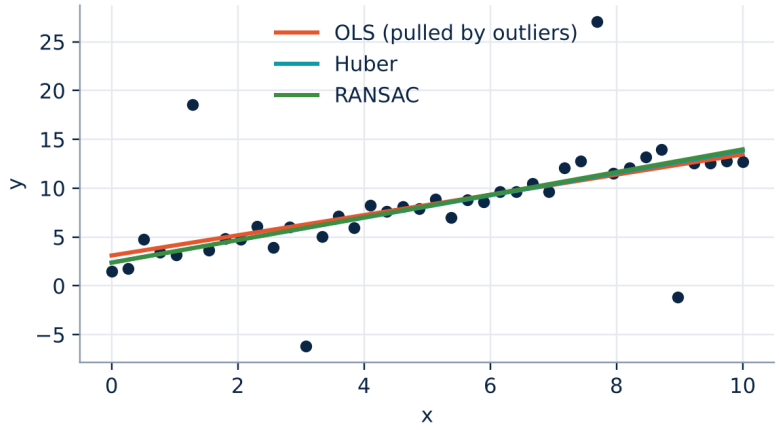


Figure 8. Four outliers pull the OLS line away from the trend, while Huber and RANSAC stay on it.

PYTHON Contaminate the target and compare robust models

```python
from sklearn.linear_model import HuberRegressor, RANSACRegressor, TheilSenRegressor
rng = np.random.RandomState(0)
y_bad = y_train.values.copy()
idx = rng.choice(len(y_bad), 20, replace=False)
y_bad[idx] += rng.choice([-1, 1], 20) * 400
models = {'OLS': LinearRegression(), 'Huber': HuberRegressor(max_iter=1000),
          'Theil-Sen': TheilSenRegressor(random_state=0), 'RANSAC':
              RANSACRegressor(random_state=0)}
for name, m in models.items():
    pipe = make_pipeline(StandardScaler(), m).fit(X_train, y_bad)
    print(f'{name:10s} test R2 = {r2_score(y_test, pipe.predict(X_test)):.3f}')
```

OUTPUT

```
OLS        test R2 = 0.349
Huber      test R2 = 0.423
Theil-Sen  test R2 = 0.386
RANSAC     test R2 = 0.123
```

Code walkthrough

Lines	What the code does and why
1–5	Corrupt 20 of the 353 training targets by ±400 to simulate measurement errors. The test set stays clean.
6–10	Train each model on the corrupted data and score on clean test data. Huber degrades least, then Theil-Sen; OLS loses the most among the three. RANSAC does worst here: with 10 features and 6% contaminated rows its random subsets rarely find a clean consensus set, a reminder to tune `min_samples` and `residual_threshold`.

8.5 Quantile Regression

OLS predicts the conditional **mean**. Quantile regression predicts any conditional percentile (the median, the 10th or 90th) by minimizing the asymmetric **pinball loss**. It needs no normality assumption, is robust to outliers (the median version) and gives prediction intervals when you fit two quantiles.

$$\rho_\tau(e) = \tau e \ \text{ if } e \geq 0, \quad (\tau - 1)e \ \text{ if } e < 0 \qquad \hat{\boldsymbol{\beta}}_\tau = \arg\min_{\boldsymbol{\beta}} \sum_i \rho_\tau\big(y_i - \mathbf{x}_i^T \boldsymbol{\beta}\big)$$

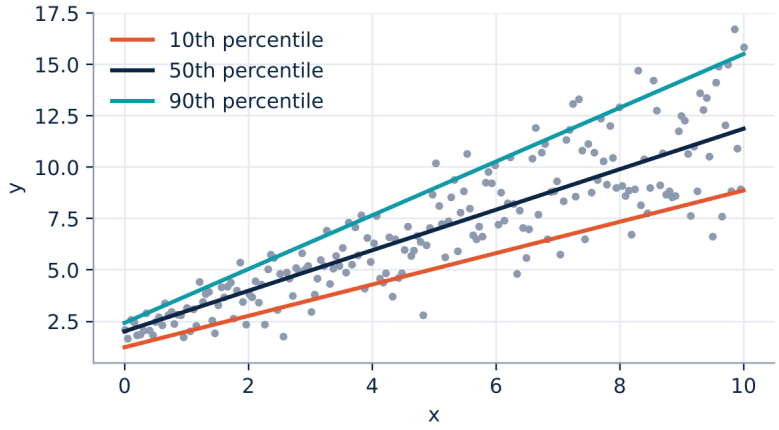


Figure 9. 10th, 50th and 90th percentile lines fan out where the data's spread grows.

PYTHON Fit three quantiles and check their coverage

```python
from sklearn.linear_model import QuantileRegressor
for q in (0.1, 0.5, 0.9):
    m = make_pipeline(StandardScaler(), QuantileRegressor(quantile=q, alpha=0,
        solver='highs'))
    pred = m.fit(X_train, y_train).predict(X_test)
    print(f'quantile {q}: {(y_test <= pred).mean():.0%} of test targets fall below the
        prediction')
```

OUTPUT

```
quantile 0.1: 9% of test targets fall below the prediction
quantile 0.5: 45% of test targets fall below the prediction
quantile 0.9: 89% of test targets fall below the prediction
```

Code walkthrough

Lines	What the code does and why
1–5	Fit one model per quantile (`alpha=0` turns off the L1 penalty). A well-calibrated q-quantile model should have roughly q × 100% of the observations below its predictions.

8.6 LARS, LassoLars and Orthogonal Matching Pursuit

- **LARS (Least Angle Regression):** adds features one at a time, moving coefficients in the direction equiangular to the residual correlations. Efficient when p n and produces the whole solution path.
- **LassoLars:** a LARS variant that computes the exact Lasso path quickly.
- **Orthogonal Matching Pursuit (OMP):** greedy forward selection that stops at exactly *k* non-zero coefficients, used in signal processing and sparse coding.

PYTHON Three sparse solvers

```python
from sklearn.linear_model import Lars, LassoLarsCV, OrthogonalMatchingPursuitCV
solvers = {'LARS (6 features)': Lars(n_nonzero_coefs=6),
           'LassoLarsCV': LassoLarsCV(cv=5),
           'OMP (CV)': OrthogonalMatchingPursuitCV(cv=5)}
for name, m in solvers.items():
    pipe = make_pipeline(StandardScaler(), m).fit(X_train, y_train)
    nz = int((pipe[-1].coef_ != 0).sum())
    print(f'{name:18s} features used = {nz:2d}   test R2 = {r2_score(y_test,
        pipe.predict(X_test)):.3f}')
```

OUTPUT

```
LARS (6 features)  features used =  6   test R2 = 0.469
LassoLarsCV        features used =  7   test R2 = 0.471
OMP (CV)           features used =  5   test R2 = 0.469
```

Code walkthrough

Lines	What the code does and why
1–4	Create the three estimators: LARS capped at 6 features, and two variants that choose their sparsity by cross-validation.
5–8	Fit each solver, count the non-zero coefficients and report accuracy: sparse models reach the accuracy of the full model with fewer features.

8.7 SGDRegressor (Stochastic Gradient Descent)

Updates the weights using one sample (or a mini-batch) at a time, so it scales to millions of rows and supports streaming and out-of-core learning. Choose the loss (squared, Huber, epsilon-insensitive) and penalty (L2, L1, ElasticNet) freely. **Feature scaling is essential.**

PYTHON Linear regression trained by stochastic gradient descent

```python
from sklearn.linear_model import SGDRegressor
sgd = make_pipeline(StandardScaler(),
    SGDRegressor(loss='squared_error', penalty='l2', alpha=1e-3,
                 learning_rate='invscaling', eta0=0.01, max_iter=2000, tol=1e-4,
                     random_state=0))
sgd.fit(X_train, y_train)
print('epochs used:', sgd[-1].n_iter_)
print('test R2   :', round(r2_score(y_test, sgd.predict(X_test)), 3))
```

OUTPUT

```
epochs used: 30
test R2   : 0.456
```

Code walkthrough

Lines	What the code does and why
1–5	`loss` and `penalty` select the objective; `eta0` is the starting learning rate, decayed by the `invscaling` schedule; training stops when improvement falls below `tol`.
6–7	SGD converges to nearly the same solution as the closed-form model.

8.8 Principal Component Regression and Partial Least Squares

- **PCR:** run PCA on X, keep the top *k* orthogonal components and regress y on them. Removes multicollinearity but ignores y when choosing components.
- **PLS:** finds components that maximize covariance with y, so the components are chosen for predictive power. Popular in chemometrics and spectroscopy where p n.

PYTHON PCR vs PLS

```python
from sklearn.decomposition import PCA
from sklearn.cross_decomposition import PLSRegression
pcr = make_pipeline(StandardScaler(), PCA(n_components=6), LinearRegression()).fit(X_train,
    y_train)
pls = make_pipeline(StandardScaler(), PLSRegression(n_components=3)).fit(X_train, y_train)
print('PCR variance kept:', round(pcr[1].explained_variance_ratio_.sum(), 3))
print('PCR test R2:', round(r2_score(y_test, pcr.predict(X_test)), 3))
print('PLS test R2:', round(r2_score(y_test, pls.predict(X_test).ravel()), 3))
```

OUTPUT

```
PCR variance kept: 0.896
PCR test R2: 0.438
```

```
PLS test R2: 0.46
```

Code walkthrough

Lines	What the code does and why
1–4	Build both pipelines: PCA then OLS, versus PLS with only three components.
5–7	Six principal components keep most of the variance of X. PLS reaches a similar R^2 with fewer components because they are supervised.

8.9 Generalized Linear Models: Poisson, Gamma and Tweedie

GLMs relax the normal-error assumption. A **link function** g connects the linear predictor to the mean of a distribution from the exponential family, which keeps predictions in a valid range (for example, always positive).

$$g(\mathbb{E}[y \mid \mathbf{x}]) = \mathbf{x}^T\boldsymbol{\beta} \qquad \text{e.g. log link: } \mathbb{E}[y] = e^{\mathbf{x}^T\boldsymbol{\beta}}$$

Model	Target type	Distribution / link	Example
Poisson	Counts 0,1,2…	Poisson, log link	Number of insurance claims
Gamma	Positive, skewed continuous	Gamma, log link	Claim size, waiting time
Tweedie	Zero-inflated positive	Compound Poisson–Gamma	Insurance pure premium
Logistic	Binary 0/1 (classification)	Binomial, logit link	Default or no default

PYTHON Gamma and Poisson GLMs

```python
from sklearn.linear_model import TweedieRegressor, PoissonRegressor
gamma = make_pipeline(StandardScaler(), TweedieRegressor(power=2, link='log', alpha=0.01,
    max_iter=2000)).fit(X_train, y_train)
pois  = make_pipeline(StandardScaler(), PoissonRegressor(alpha=0.01,
    max_iter=2000)).fit(X_train, y_train)
print('Gamma  D2 score:', round(gamma.score(X_test, y_test), 3), '  min prediction:',
    round(gamma.predict(X_test).min(), 1))
print('Poisson D2 score:', round(pois.score(X_test, y_test), 3), '  min prediction:',
    round(pois.predict(X_test).min(), 1))
```

OUTPUT

```
Gamma  D2 score: 0.445   min prediction: 69.2
Poisson D2 score: 0.467   min prediction: 69.0
```

Code walkthrough

Lines	What the code does and why
1–3	`TweedieRegressor(power=2)` is the Gamma GLM; power=1 is Poisson, power=0 is the normal model. The log link guarantees positive predictions. This target is positive, so both are valid choices.
4–5	GLMs report the **D^2 score** (fraction of deviance explained), the GLM analogue of R^2. Notice every prediction is positive.

8.10 Kernel Ridge Regression

Ridge regression in a (possibly infinite-dimensional) feature space defined by a **kernel** such as the RBF. It captures smooth non-linear relationships without manually creating polynomial features, but training cost grows as $O(n^3)$ so it suits small to medium datasets.

PYTHON RBF Kernel Ridge with a small grid search

```python
from sklearn.kernel_ridge import KernelRidge
from sklearn.model_selection import GridSearchCV
grid = GridSearchCV(KernelRidge(kernel='rbf'),
    {'alpha': [0.1, 1, 10], 'gamma': [0.003, 0.01, 0.03]}, cv=5, scoring='r2')
kr = make_pipeline(StandardScaler(), grid).fit(X_train, y_train)
print('best params:', grid.best_params_)
print('test R2    :', round(r2_score(y_test, kr.predict(X_test)), 3))
```

OUTPUT

```
best params: {'alpha': 0.1, 'gamma': 0.003}
test R2    : 0.48
```

Code walkthrough

Lines	What the code does and why
1–5	Search the regularization strength `alpha` and the kernel width `gamma` with cross-validation inside the pipeline.
6–7	Kernel Ridge usually matches or beats linear models when the true relationship is mildly non-linear.

8.11 Feature selection: RFE and stepwise methods

Recursive Feature Elimination (RFE) repeatedly fits the model, removes the weakest feature (smallest |coefficient|) and repeats, using cross-validation to decide how many features to keep. Classical **forward/backward stepwise** selection (available as `SequentialFeatureSelector`) adds or removes one feature at a time to improve a score.

PYTHON RFECV to find the best feature subset

```python
from sklearn.feature_selection import RFECV
rfe = RFECV(LinearRegression(), step=1, cv=5, scoring='r2')
rfe.fit(StandardScaler().fit_transform(X_train), y_train)
print('optimal number of features:', rfe.n_features_)
print('selected:', list(X.columns[rfe.support_]))
```

OUTPUT

```
optimal number of features: 8
selected: ['sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5']
```

Code walkthrough

Lines	What the code does and why
1–3	RFECV eliminates one feature per round and scores every subset size with 5-fold CV. Data is scaled first so coefficient sizes are comparable.
4–5	`support_` is a boolean mask of the kept features; indexing the column names with it lists them.

8.12 Other specialised linear models

Model	Idea	Typical use
MultiTaskLasso / MultiTaskElasticNet	Lasso shared across several targets so the same features are chosen for all	Multi-output problems
Adaptive Lasso	Weighted L1 penalty with weights from an initial OLS/Ridge fit; fixes Lasso's selection bias	Consistent variable selection
Group Lasso	Penalizes groups of coefficients together (all-or-nothing)	Categorical features expanded by one-hot encoding
Fused Lasso	Penalizes differences between neighbouring coefficients	Time series, spatial and genomic data
Passive-Aggressive Regressor	Online learner that updates only when the error exceeds a margin	Streaming data
Spline / GAM regression	Piecewise polynomials with smooth joins; additive non-linear terms	Smooth non-linear effects with interpretability
LOESS / local regression	Fits small weighted lines around every query point	Exploratory smoothing
Total least squares (Deming)	Minimizes errors in both x and y	Measurement-error problems
Isotonic regression	Fits a monotonic non-decreasing step function	Calibration, dose-response curves

9. Model Comparison and How to Choose

All models below were trained on the same 353 training rows of the Diabetes dataset (10 features) and evaluated on the same 89 held-out rows, with standardized inputs. CV R^2 is the 5-fold cross-validated score on the training data.

Rank	Model	CV R^2	Test R^2	Test RMSE	Test MAE
1	Polynomial (deg 2) + Ridge	0.447	0.510	50.95	40.44
2	Kernel Ridge (RBF)	0.470	0.482	52.40	42.05
3	Lasso (CV)	0.477	0.471	52.92	42.79
4	LassoLars	0.477	0.471	52.92	42.79
5	Orthogonal Matching Pursuit	0.474	0.469	53.02	42.81
6	LARS	0.469	0.469	53.06	43.23
7	ARD Regression	0.481	0.467	53.14	42.73
8	ElasticNet (CV)	0.477	0.461	53.44	42.95
9	PLS Regression	0.478	0.460	53.48	42.88
10	Bayesian Ridge	0.479	0.458	53.59	42.87
11	SGD Regressor	0.478	0.456	53.69	42.90
12	Ridge (CV)	0.481	0.454	53.77	42.81
13	OLS	0.480	0.453	53.85	42.79
14	Gamma GLM (log link)	0.465	0.452	53.91	41.89
15	Huber	0.473	0.447	54.14	42.88
16	Theil-Sen	0.472	0.443	54.32	43.17
17	PCR (PCA + OLS)	0.477	0.438	54.57	43.84
18	Quantile (median)	0.460	0.422	55.32	43.45
19	RANSAC	-0.116	0.085	69.61	56.31

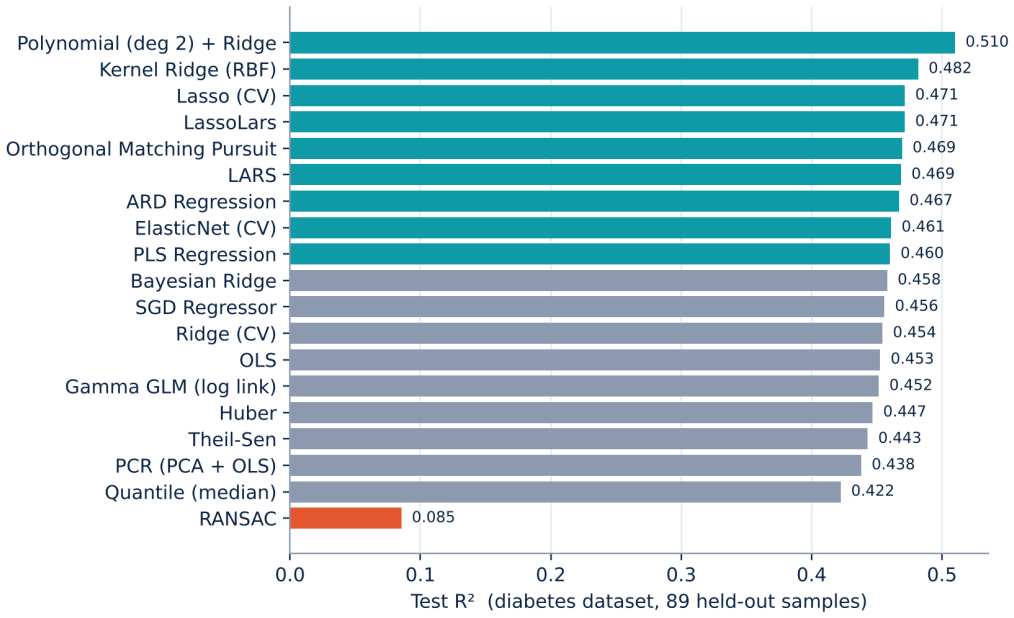


Figure 10. Test R^2 of every model (teal = strong, grey = average, orange = weak).

What the numbers tell us

Most linear models land within a narrow band ($R^2 \approx 0.42$–0.48), so on this small, mostly linear dataset **model choice matters less than good preprocessing and validation**.
Regularized and sparse models (Lasso, LARS, OMP) match OLS with fewer active features, while mildly non-linear models (Polynomial + Ridge, Kernel Ridge) edge ahead. **RANSAC** performs poorly here because there are no real outliers and its random inlier subsets throw away useful data.
Differences of 0.01–0.02 in R^2 on 89 test rows are within noise; rely on cross-validation, not one split.

9.1 Decision guide

Situation	Recommended model
Few features, clean data, need interpretation and p-values	OLS (statsmodels)
Correlated features, want stable predictions	Ridge
Many features, expect only a few to matter	Lasso (or LassoLars / OMP for speed)
Many correlated features and want selection	ElasticNet
Curved relationship	Polynomial + Ridge, spline/GAM, Kernel Ridge
Outliers in the target	Huber (first choice), Theil-Sen, RANSAC, median Quantile regression
Need prediction intervals or percentiles	Quantile regression, Bayesian Ridge
Unequal variance or correlated errors	WLS, GLS / GLSAR
Millions of rows, streaming data	SGDRegressor
p much larger than n, collinear predictors	PLS, PCR, ElasticNet
Count or strictly positive target	Poisson, Gamma or Tweedie GLM

9.2 Master comparison of the penalized family

Property	OLS	Ridge	Lasso	ElasticNet
Penalty	None	L2	L1	L1 + L2
Feature selection	No	No	Yes	Yes
Multicollinearity	Poor	Excellent	Moderate	Excellent
When p > n	Fails	Works	Selects at most n	Works
Closed form	Yes	Yes	No (coordinate descent)	No (coordinate descent)
Hyper-parameters	0	1 (alpha)	1 (alpha)	2 (alpha, l1_ratio)
sklearn class	LinearRegression	Ridge / RidgeCV	Lasso / LassoCV	ElasticNet / ElasticNetCV

10. Cheat Sheet and Summary

Topic	Remember
Model	$\hat{y} = X\beta$; OLS minimizes the sum of squared residuals; $\beta = (X^TX)^{-1}X^Ty$
Assumptions	Linearity, Independence, Normality, Equal variance, no multicollinearity
Metrics	MAE, MSE, RMSE (error size); R^2 and adjusted R^2 (variance explained); always on test data
Overfitting	Many or correlated features → add a penalty (Ridge/Lasso/ElasticNet) and tune λ by cross-validation
Preprocessing	Split first, then scale inside a Pipeline to avoid leakage
Outliers	Huber, RANSAC, Theil-Sen, quantile (median) regression
Non-linear	Polynomial features, kernels, splines
Non-normal target	GLMs: Poisson (counts), Gamma (positive skewed)
Interpretation	Coefficients show association, not causation; do not extrapolate outside the data range

Learning roadmap

- Re-implement simple and multiple regression from scratch (Sections 2–3).
- Practise diagnostics with statsmodels on your own dataset (Section 4).
- Compare Ridge, Lasso and ElasticNet with cross-validation (Section 7).
- Try robust, quantile and GLM models when assumptions fail (Section 8).
- Package the final model as a Pipeline and save it with `joblib` (Section 6).

About the author

These notes were prepared by **Chandan Chaudhari**.

GitHub: github.com/chandanc5525

Feedback, corrections and stars on the repository are always welcome.